Weekly Project Progress Report

Team Name: Quick Bite

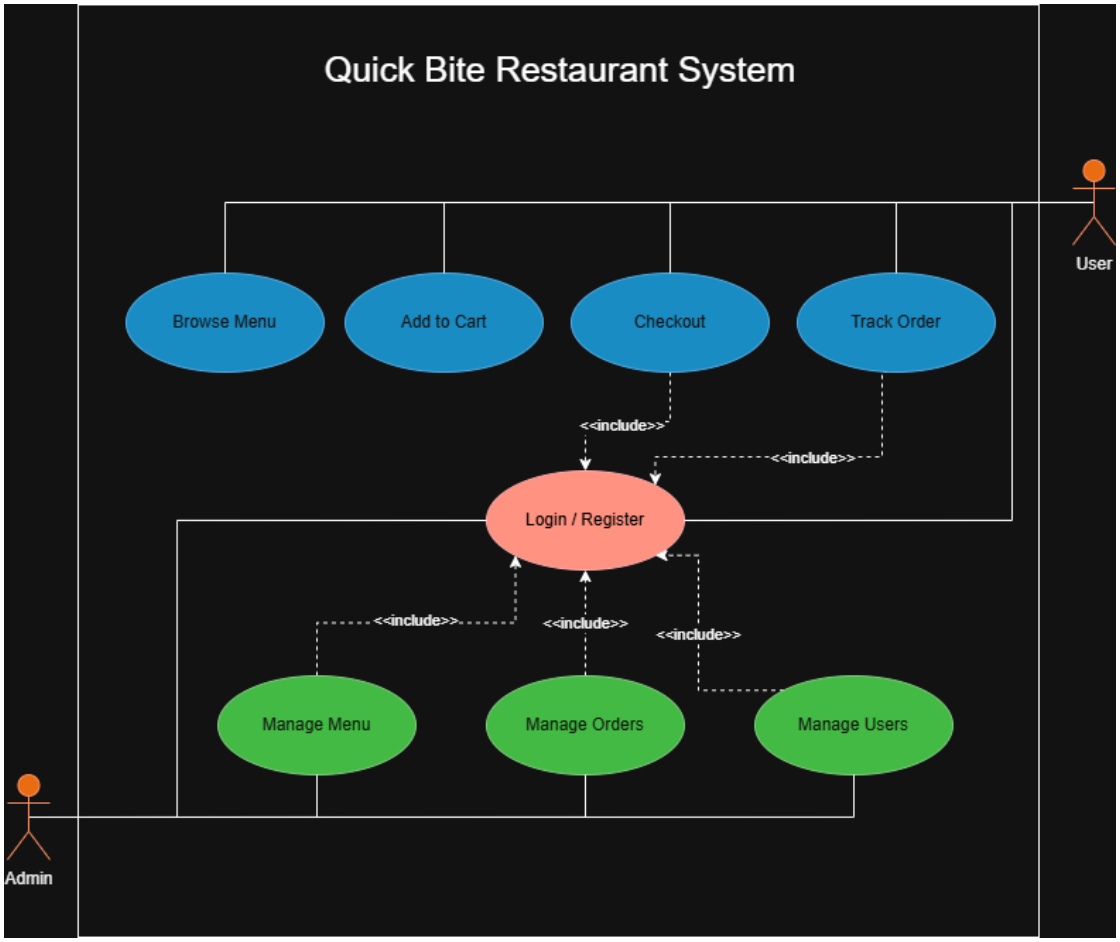
Week Number: Week 3 – System Architecture Design

Team Members:

1. Bahaa Eddin Said Al-Mallahi 120210277
2. Abdullah Mohammed AbuZaid 120211559
3. Abdullah Ali Al-Hindawi 120211863
4. Abdalkareem Rajab Abo Younis 120211877
5. Hazem Mohamed Oukal 120212771

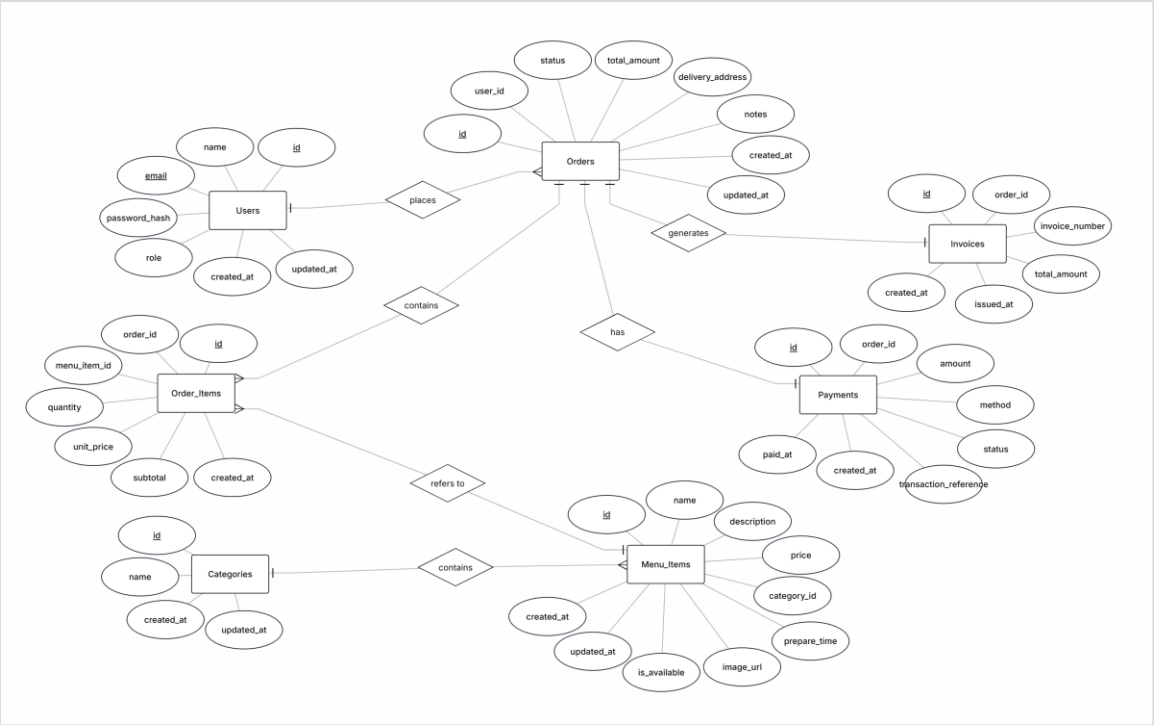
Work completed this week

We completed the system architecture design for our project, "Quick Bite Restaurant System." This week, we designed the Use Case Diagram using two main actors: User and Admin. We also created the ERD to define the main database tables and relationships. In addition, we identified the main system processes and selected the initial technology stack for the project.



Use Case Diagram

The Use Case Diagram illustrates the main interactions in the Quick Bite Restaurant System.
The system was simplified into two main actors: User and Admin.
The User can browse the menu, add items to the cart, checkout, and track orders.
The Admin manages menu items, orders, and users.
Authentication is required for checkout, order tracking, and admin operations.



Database Schema (ERD)

The ERD represents the main database structure of the Quick Bite Restaurant System.
It includes the core tables: Users, Orders, Order_Items, Menu_Items, Categories, Payments, and Invoices.
The relationships between these tables were designed to support menu browsing, order creation, payment handling, and invoice generation.

Tech Stack

The project will use the following technologies:

- **Frontend:** React
  React is a JavaScript library for building user interfaces using reusable components. It improves performance through the virtual DOM and is used in this project to create a responsive and interactive frontend.

- **Backend:** Python (FastAPI Framework)

  FastAPI is planned to be used as the backend framework for the project. It is known for its high performance and ease of use, relying on Pydantic and Starlette to make it fast and efficient when handling data.

  Key features of FastAPI:

  High Speed: It is one of the fastest frameworks for building APIs, outperforming most others like Flask and Django.

  Data Validation: FastAPI uses Pydantic for easy data validation, reducing errors and improving reliability.

  Automatic Documentation: It provides automatic API documentation using Swagger and ReDoc, making it easier for developers to test and understand the API endpoints.

  Easy Integration: It integrates well with various databases and technologies like SQLAlchemy, PostgreSQL, and more.

  In summary, FastAPI is an excellent choice for developing high-performance, reliable web applications.

- **Database:** MySQL

  MySQL is planned to be used as the primary relational database management system for the project. It will be responsible for storing and organizing structured data efficiently, including users, orders, menu items, payments, and invoices. The database design supports core operations such as data insertion, retrieval, updating, and deletion (CRUD). MySQL also ensures data integrity and consistency through relationships and constraints between tables. This makes it suitable for handling the backend data needs of the Quick Bite Restaurant System.

- **Version Control:** GitHub
  GitHub is a platform for version control and team collaboration, used to manage and track project code.

- **Design Tools:** draw.io and ERDPlus
  draw.io was used to design system diagrams, including the Use Case Diagram.

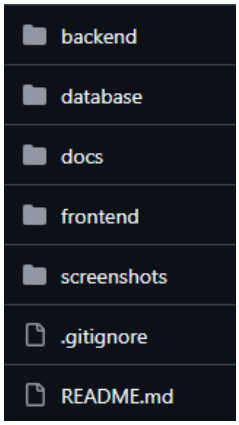
  ERDPlus was used to design the database structure, including the ERD.

Project Implementation / GitHub Repository

- **GitHub Repository Link:**

  https://github.com/AbdullahAbuZaid04/quick-bite-restaurant.git

- **Structure Image:**



Features implemented

The following features were identified and designed this week:

- User registration and login
- Browse menu
- Add items to cart
- Checkout
- Track order
- Admin menu management
- Admin order management
- Admin user management

Problems encountered

We faced some difficulty in deciding whether to use two actors or three actors in the Use Case Diagram. We also had some confusion while defining the relationships in the ERD, especially between orders, payments, and invoices. In addition, we needed to simplify the overall system design to keep it suitable for the project scope.

Solutions applied

To solve these issues, we simplified the system design by using two actors only: User and Admin. We combined the management responsibilities into the admin actor to make the diagram clearer and easier to understand. We also revised the ERD and kept the database schema simple, with clearly defined tables and relationships.

Plan for next week

Next week, we will design UI wireframes for the main pages and create a user flow diagram to represent the navigation within the system. We will also update the GitHub repository.

Questions for instructor

No questions at this stage